

School of Computer Science and Engineering

RV University

July, 2026

ROADSENSE: ROAD INFRASTRUCTURE MONITORING USING MULTI-SENSOR FUSION

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology (Honours)

by

SAKET M

USN: 1RUA24CSE0395

Guided by

Prof. Chidhananda R S

Assistant Professor, SoCSE, RV University

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

School of Computer Science and Engineering

RV University

RV Vidyaniketan, 8th Mile, Mysore Road, Bengaluru, Karnataka, India - 560059

CERTIFICATE

This is to certify that the project work titled "RoadSense: Road Infrastructure Monitoring Using Multi-Sensor Fusion" has been carried out by Saket M, bearing USN: 1RUA24CSE0395, a bonafide student of the B.Tech (Honours) programme at the School of Computer Science and Engineering, RV University, Bengaluru, in partial fulfillment of the requirements for the award of the degree of B.Tech (Honours) in Computer Science and Engineering during the academic year 2026-2027.

Name of the Examiner	Signature of Examiner
1.	1.
2.	2.

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

DECLARATION

I hereby declare that the thesis entitled "RoadSense: Road Infrastructure Monitoring Using Multi-Sensor Fusion" submitted by me, for the award of the degree B.Tech (Honours) in RV University is a record of Bonafide work carried out by me under the supervision of Prof. Chidhananda R S.

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Bangalore

Date:

Saket M

ABSTRACT

Road infrastructure in Indian cities deteriorates quickly due to monsoons, heavy traffic, and poor maintenance. Government surveys are expensive and slow, which means that dangerous potholes stay unpatched for months. This project presents RoadSense, a low-cost system that detects and maps road damage automatically. A smartphone mounted on a two-wheeler captures video and accelerometer readings at the same time. The video is fed to a YOLOv8 convolutional neural network that spots potholes visually, while the accelerometer data is passed through a statistical anomaly detector that flags physical jolts. The two streams are merged using a strict verification gate: a camera detection is accepted only when the accelerometer confirms a bump at the same moment, and a severe accelerometer spike is logged even if the camera missed the hazard. This sensor-fusion approach cut false positives by over ninety percent compared to camera-only detection. Training the neural network raised a challenge known as catastrophic forgetting; retraining on shadow images alone made the model go blind to every object. The problem was solved by building a mixed dataset that contained both genuine pothole photographs sourced from Roboflow and locally extracted shadow frames, then retraining on a free Kaggle cloud GPU. The final output is a lightweight CSV file of verified GPS coordinates which is rendered on a dark-mode Leaflet map inside a serverless progressive web application hosted at zero cost. In field trials on four separate rides across Bengaluru, the pipeline correctly identified thirty-five verified potholes from over five thousand video frames and three hundred sixty sensor readings, proving that commodity hardware and open-source models can replace expensive survey equipment.

ACKNOWLEDGMENT

I would like to express my heartfelt gratitude to Prof. Chidhananda R S, Assistant Professor, School of Computer Science and Engineering, RV University, for offering invaluable support, expert guidance, and constant encouragement throughout the course of this project. Their insights and constructive feedback greatly contributed to the successful completion of this work. It has been a privilege to work under the mentorship of such a knowledgeable and experienced professional in the field of computer vision and IoT systems.

I would like to express my gratitude to Dr Ram Kumar Kakani, Vice Chancellor RV University, and Dr Shobha G, Dean SoCSE, RV University, for providing with an environment to work in and for the inspiration during the tenure of the course.

I express my sincere gratitude to Prof. Narendra, Program Director, B.Tech (H), School of Computer Science & Engineering, RV University, for the guidance and support provided throughout my course of study. I also extend my heartfelt thanks to all the faculty members and staff of the School for their constant encouragement, valuable guidance, and assistance, which contributed significantly to the successful completion of my studies. Finally, I would like to express my deepest gratitude to my parents for their unwavering support, encouragement, and belief in me throughout this journey.

I would like to express my sincere gratitude to my mentor, Prof. Chidhananda R S, Assistant Professor, School of Computer Science and Engineering, RV University, for their guidance, support, and valuable suggestions throughout the course of this project. Their constructive feedback and encouragement played an important role in the successful completion of this work. I am grateful for the time and effort they dedicated to mentoring me during this project.

It is indeed a pleasure to thank my friends who persuaded and encouraged me to take up and complete this task. At last but not least, I express my gratitude and appreciation to all

those who have helped me directly or indirectly toward the successful completion of this project.

Place: Bangalore

Date:

Saket M

CONTENTS

ABSTRACT i

ACKNOWLEDGMENT ii

CONTENTS iii

LIST OF FIGURES iv

LIST OF TABLES v

LIST OF ACRONYMS vi

CHAPTER 1: INTRODUCTION

1.1 Introduction 1

1.2 Aim and Objectives 3

1.3 Problem Statement 4

1.4 Scope of the Project 5

1.5 Challenges 6

CHAPTER 2: BACKGROUND

2.1 Literature Survey 8

2.2 Requirements 12

CHAPTER 3: METHODOLOGY

3.1 Proposed Methodology 14

3.2 Datasets 17

3.3 Flow of the Model 19

CHAPTER 4: RESULTS

4.1 Experimental Setup 22

4.2 Result Summary Tables 24

4.3 Analysis of Results 25

4.4 Comparison with Existing Methods 27

CHAPTER 5: INFERENCES

5.1 Key Findings 29

5.2 Interpretation of Results 30

5.3 Implications of the Study 31

5.4 Fulfillment of Objectives 32

CHAPTER 6: CONCLUSION

6.1 Results 34

6.2 Future Work 35

6.3 References 37

APPENDIX 38

LIST OF FIGURES

Figure 1.1: RoadSense System Architecture Overview 2

Figure 1.2: Hardware Setup on Two-Wheeler 3

Figure 3.1: End-to-End Pipeline Flowchart 15

Figure 3.2: YOLOv8 Detection on Road Frames 16

Figure 3.3: Sensor Fusion Verification Gate Logic 18

Figure 3.4: Mixed Dataset Composition 19

Figure 4.1: Detection Count Progression Across Training Runs ... 23

Figure 4.2: Dark-Mode Map Rendering with Pothole Overlay 26

Figure 5.1: False Positive Reduction Through Sensor Fusion 30

LIST OF TABLES

Table 2.1: Comparison of Existing Road Monitoring Systems 10

Table 3.1: Mixed Dataset Composition 17

Table 4.1: Detection Results Across Four Training Stages 24

Table 4.2: Sensor Fusion Verification Breakdown (test/four) 25

Table 5.1: Statistical Validation of Objectives 32

LIST OF ACRONYMS

YOLO	You Only Look Once
CNN	Convolutional Neural Network
IoT	Internet of Things
GPS	Global Positioning System
IMU	Inertial Measurement Unit
CSV	Comma Separated Values
OSM	OpenStreetMap
PWA	Progressive Web Application
API	Application Programming Interface
GPU	Graphics Processing Unit
SaaS	Software as a Service
UI	User Interface

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

India has one of the largest road networks on the planet, stretching well over six million kilometres. These roads carry everything from school buses to freight trucks, and their condition directly shapes the safety and economic productivity of the communities they serve. Yet a staggering proportion of these roads are plagued by surface defects. Potholes, cracks, and uneven patches develop rapidly during the monsoon season when rainwater seeps through cracks in the asphalt and weakens the sub-base layer. Heavy commercial vehicles then pound that weakened surface until it collapses into a cavity. The National Crime Records Bureau has repeatedly noted that poor road conditions contribute to a significant share of traffic fatalities every year, making road damage not just an inconvenience but a genuine public-safety crisis.

Despite the severity of the problem, the way road damage is detected and catalogued has barely changed in decades. Municipal authorities typically rely on manual inspections: an engineer rides through a ward, notes down the locations of visible defects, and files a written report that slowly works its way through the bureaucracy. This approach is labour-intensive, subjective, and painfully slow. By the time a pothole is officially recorded, patched, and inspected again, weeks or even months may have passed. During that window, motorcyclists swerve to avoid it, sometimes into oncoming traffic, and ambulances bounce through it, sometimes with critical patients on board.

In recent years, computer vision has emerged as a promising alternative. Convolutional neural networks, particularly the YOLO family of models, can be trained to spot road defects in video frames with high accuracy. At the same time, smartphones have become extraordinarily powerful sensor platforms: a mid-range Android handset now ships with a three-axis accelerometer, a gyroscope, a barometer, a high-resolution camera, and a precise GPS receiver. The idea of strapping a phone to a vehicle and letting software do

the surveying is not new, but most existing systems treat the camera and the accelerometer as independent channels. They either analyse video or analyse vibrations, rarely both at the same time, and almost never cross-check one against the other.

RoadSense was built to close that gap. It is an end-to-end pipeline that records video and inertial data simultaneously, runs a YOLOv8 object-detection model on the stabilised video, computes statistical anomaly scores from the accelerometer trace, and then fuses the two signals through a strict verification gate. A visual detection is trusted only if the accelerometer confirms a physical disturbance at the same timestamp; conversely, a violent physical jolt is logged even if the camera happened to be blurred at that instant. The verified detections are written to a lightweight CSV file, which a serverless web application renders on an interactive dark-mode map. The entire system runs on commodity hardware, a phone taped to a scooter handlebar, uses only free and open-source software, and costs nothing to host.

1.2 AIM AND OBJECTIVES

The primary aim of this project is to design and develop an affordable, automated road-condition monitoring system that fuses visual and inertial sensor data to map potholes with high accuracy and minimal false positives.

The specific objectives are as follows:

1. To build a data-collection workflow that simultaneously captures dashcam video and time-stamped accelerometer/GPS readings from a smartphone mounted on a two-wheeler.

2. To stabilise the recorded video using frame-to-frame optical-flow estimation and affine-transform correction, thereby reducing motion-induced blur before feeding frames to the detection model.

3. To train and fine-tune a YOLOv8 convolutional neural network for visual pothole detection, addressing the challenge of catastrophic forgetting by constructing a mixed dataset of genuine pothole images and locally extracted shadow frames.

4. To develop a speed-adaptive physical anomaly detector that computes Z-scores from the accelerometer trace relative to a per-vehicle calibration profile, enabling the system to distinguish potholes from normal road vibrations at any speed.

5. To implement a strict sensor-fusion verification gate that cross-validates camera detections against accelerometer evidence, accepting a visual detection only when the physics confirm a bump and overriding the camera with a sensor-only flag when a severe jolt is recorded.

6. To deliver a lightweight, zero-cost web application that renders the verified pothole coordinates on an interactive map, allowing municipal authorities and citizens to visualise road quality at a glance.

1.3 PROBLEM STATEMENT

Manual road surveys are expensive, slow, and subjective. Camera-only detection systems are susceptible to false positives triggered by shadows, paint markings, and wet patches. Accelerometer-only systems cannot distinguish a pothole from a speed bump or a road joint. There is a clear need for a low-cost, fully automated system that fuses visual and physical sensor data to reliably detect and geo-locate road surface defects while suppressing false alarms.

1.4 SCOPE OF THE PROJECT

The scope of this project covers the full lifecycle of a road-condition survey: data collection on a moving two-wheeler, video stabilisation, AI-based visual detection, physical anomaly scoring, sensor fusion, and web-based visualisation. The system is designed for urban and semi-urban roads in Indian cities and has been validated on four separate rides across Bengaluru. It does not currently cover highways with concrete surfaces, nor does it attempt to classify defect severity beyond a binary pothole-or-not label. The web application is a client-side static site and does not include a cloud-hosted backend for video processing.

1.5 CHALLENGES

Several significant challenges were encountered during development:

Shadow Interference: Late-afternoon sunlight casts long, dark shadows that visually resemble potholes. The untrained YOLO model flagged 192 shadow-based false detections in a single five-minute ride, making the output practically useless for any real deployment.

Catastrophic Forgetting: When the model was retrained exclusively on shadow images to suppress false positives, it lost the ability to detect real potholes entirely. This phenomenon, well documented in the transfer-learning literature, reduced visual detections from 192 to zero and required a fundamentally different training strategy.

Dataset Scarcity: No single public dataset contained both pothole-positive and shadow-negative images in the proportions needed. The solution was to merge thousands of labelled pothole frames from Roboflow with fifty-five locally extracted shadow frames, creating a custom mixed dataset.

Compute Constraints: Training a YOLOv8 model on a consumer laptop without a discrete GPU would have taken many hours. The project leveraged free Kaggle cloud GPUs to bring training time down to minutes.

GPS Cold-Start Drift: Smartphone GPS receivers often report coordinates of (0, 0) for the first few seconds after initialisation. These spurious readings stretched the map bounding box to the coast of West Africa, requiring a coordinate-validation filter in the web application.

CHAPTER 2

BACKGROUND

2.1 LITERATURE SURVEY

Road-surface monitoring has attracted steady research interest over the past decade. Early systems relied on dedicated hardware: laser profilometers and high-speed cameras mounted on survey vans that cost hundreds of thousands of dollars. While accurate, these rigs were impractical for developing countries where municipal budgets are tight and the road network is vast. Researchers therefore turned to commodity sensors, particularly smartphone accelerometers and cameras, as low-cost substitutes.

Eriksson et al. (2008) introduced Pothole Patrol, one of the first systems to use smartphone accelerometers for pothole detection. The team mounted phones in taxis across Boston and trained a supervised classifier on hand-labelled vibration windows. The approach worked well on smooth urban asphalt but struggled with cobblestone streets and speed bumps, which produced accelerometer signatures nearly identical to those of potholes. The core limitation was that vibrations alone carry no information about the visual appearance of the road surface.

Mednis et al. (2011) extended the accelerometer approach by experimenting with different threshold-based algorithms on Z-axis readings. They compared a simple peak detector with a Z-score method and found that the Z-score approach adapted better to varying vehicle speeds. However, their system still could not tell a pothole from a railway crossing or a drainage grate, because all three produce sharp vertical accelerations.

On the visual side, Maeda et al. (2018) trained a deep-learning model to detect road damage from dashboard-camera images. Their dataset, collected across Japanese municipalities, contained over nine thousand labelled images spanning cracks, potholes, and lane markings. The model achieved strong precision on well-lit, dry roads, but

performance dropped sharply in the presence of shadows, wet surfaces, and poor lighting, conditions that are common during the Indian monsoon.

Koch and Brilakis (2011) proposed a vision-based system that used shape descriptors and histogram analysis to detect potholes in still images. Their method was computationally cheap but brittle: it relied heavily on the assumption that potholes are darker than the surrounding surface, an assumption that breaks down whenever tree shadows or oil stains are present.

More recently, Arya et al. (2021) explored fusing accelerometer and camera data for pothole detection in Indian road conditions. They used a two-stage pipeline: the accelerometer identified candidate rough-road windows, and a CNN then classified still frames within those windows. However, their fusion was sequential rather than simultaneous: the accelerometer acted as a coarse filter, not as a verification mechanism. Consequently, the system could still accept a shadow frame as a pothole if the road happened to be rough at that moment for an unrelated reason.

Jo and Ryu (2015) investigated pothole detection using three-dimensional point clouds from stereo cameras. The approach was accurate in controlled settings but required calibrated stereo hardware, making it unsuitable for crowdsource-style deployment on personal vehicles.

The present project, RoadSense, addresses the limitations of prior work by implementing a strict verification gate. Rather than using sensors to merely filter candidate windows, the system cross-validates every individual detection against the physics. A camera detection is accepted only if the accelerometer confirms a bump at the same timestamp, and a severe physical event is logged regardless of the camera output. This bidirectional fusion is what distinguishes RoadSense from the sequential pipelines described in the existing literature.

2.2 REQUIREMENTS

Hardware Requirements:

1. A smartphone with a rear-facing camera capable of recording at 30 frames per second or higher.

2. A three-axis accelerometer accessible through the browser DeviceMotion API.

3. A GPS receiver with an accuracy of five metres or better.

4. A handlebar mount or adhesive cradle to secure the phone to the vehicle.

5. A laptop or desktop computer with Python 3.10 or later for running the offline processing pipeline.

Software Requirements:

1. Python 3.10+ with the following packages: ultralytics (YOLOv8), OpenCV, NumPy, pandas, and scipy.

2. A modern web browser (Chrome, Edge, Firefox, or Safari) for the RoadSense web application.

3. Leaflet.js 1.9.x for map rendering.

4. Access to a Kaggle account for free GPU-accelerated model training.

CHAPTER 3

METHODOLOGY

3.1 PROPOSED METHODOLOGY

The RoadSense pipeline is divided into four sequential stages, each of which feeds its output into the next. The overall design philosophy is edge computing: the heavy numerical work happens on a local machine, and only a tiny, verified CSV file is sent to the web for visualisation.

Stage 1 — Video Stabilisation. The raw dashcam footage from a handlebar-mounted smartphone is inherently shaky. Before any frame can be fed to the neural network, the video must be stabilised. The pipeline reads each pair of consecutive frames, converts them to greyscale, and computes a set of optical-flow feature points using the Shi-Tomasi corner detector. It then estimates an affine transformation matrix between the two frames using OpenCV. The cumulative trajectory is smoothed with a moving-average filter, and each frame is warped to remove jitter. The stabilised video is written to disk as stabilised.mp4.

Stage 2 — Visual Detection with YOLOv8. The stabilised video is loaded frame by frame and passed through a YOLOv8-nano model that has been fine-tuned on a mixed pothole-and-shadow dataset. For every frame, the model outputs zero or more bounding boxes, each with an associated confidence score. Detections below a confidence threshold of 0.5 are discarded. The remaining detections are de-duplicated using a temporal window of 0.3 seconds so that the same pothole seen across multiple consecutive frames is counted only once. The output of this stage is a list of (timestamp, confidence) tuples.

Stage 3 — Physical Anomaly Detection and Sensor Fusion. The accelerometer CSV recorded by the web application is loaded into a pandas DataFrame. For each row, the pipeline looks up the vehicle calibration profile, which stores the mean and standard

deviation of the Z-axis acceleration at each speed bin, and computes a Z-score. The fusion logic then applies two rules. First, for every visual detection from Stage 2, the pipeline checks whether the maximum Z-score in a 0.4-second window around the detection timestamp exceeds 1.5. If it does, the detection is marked as Fused and kept. If it does not, the detection is discarded as a false positive. Second, any row whose Z-score exceeds 3.0 is marked as Sensor regardless of whether the camera saw anything, because a jolt that violent almost certainly corresponds to a real surface defect.

Stage 4 — Output and Visualisation. The fused DataFrame is written to fused_labelled.csv. This file is then uploaded to the RoadSense web application, which parses it, segments the route into configurable chunks of 10, 20, or 30 metres, computes a quality rating for each segment, and draws coloured polylines on a Leaflet map. The map uses OpenStreetMap tiles with a CSS colour-inversion filter to maintain a dark aesthetic.

3.2 DATASETS

The machine-learning component of RoadSense required a dataset that contained both positive examples of potholes and negative examples of visual distractors such as shadows. No single public dataset met this requirement, so a custom mixed dataset was constructed.

The positive class was sourced from the Roboflow Pothole Detection dataset (version 3, YOLOv8 format). This dataset contains several thousand images of road surfaces with manually drawn bounding boxes around potholes. The images come from a variety of countries, lighting conditions, and camera angles, providing the diversity needed for a generalisable model.

The negative class was extracted directly from the project test rides. During a late-afternoon ride (test/four), the baseline YOLO model flagged 192 visual detections. The sensor-fusion gate later revealed that the vast majority of these detections corresponded to shadow patterns rather than genuine potholes, as confirmed by near-zero Z-scores at those timestamps. Fifty-five of the clearest shadow frames were extracted and saved with empty YOLO label files, indicating that no pothole was present.

The two classes were merged into a single training directory with the following composition:

Class	Source	Approximate Count
Pothole (Positive)	Roboflow v3	3,000+
Shadow (Negative)	Local Extraction	55
Total	Mixed	~3,055

Table 3.1: Mixed Dataset Composition

The merged dataset was uploaded to Kaggle as a zip archive and trained on a free Tesla T4 GPU. Training ran for thirty epochs using the standard YOLOv8-nano architecture with default hyperparameters.

3.3 FLOW OF THE MODEL

The complete data flow can be described in seven steps:

Step 1: The user mounts the smartphone on the vehicle handlebar and opens the RoadSense web application in a mobile browser.

Step 2: For a new vehicle, the user presses the Calibrate button and rides for thirty to sixty seconds on a road of average quality. The app records accelerometer and speed data at 10 Hz and exports a calibration CSV.

Step 3: On a laptop, the user runs "python calibrate.py <calibration_csv>" which computes the per-speed-bin mean and standard deviation of the Z-axis acceleration and saves them in vehicle_profile.json.

Step 4: The user returns to the vehicle, presses Start Recording, and rides the target route. The app simultaneously captures 30 fps rear-camera video and 2 Hz sensor telemetry, exporting a .webm video file and a .csv sensor file.

Step 5: Both files are placed in a test folder on the laptop, and the user runs "python process_ride.py test/<folder_name>". The pipeline stabilises the video, runs YOLOv8 detection, computes Z-scores, and applies the sensor-fusion gate.

Step 6: The pipeline writes fused_labelled.csv to the same folder.

Step 7: The user opens the RoadSense web app on any device, uploads the CSV, and views the colour-coded route map.

CHAPTER 4

RESULTS

4.1 EXPERIMENTAL SETUP

All experiments were conducted on real roads in Bengaluru, Karnataka, India. The test vehicle was a two-wheeler (scooter) equipped with a consumer smartphone mounted on the handlebar using a standard phone cradle. The phone ran the RoadSense web application in Google Chrome, which simultaneously captured rear-camera video at 480 x 640 resolution and 29.8 frames per second, and sensor telemetry at 2 Hz. Four separate rides were recorded, labelled test/one through test/four, covering a cumulative distance of approximately 2.5 kilometres.

The offline processing pipeline was run on a Windows laptop equipped with an Intel processor and 16 GB of RAM. No discrete GPU was available on the local machine. Model training was offloaded to a free Kaggle notebook backed by an NVIDIA Tesla T4 GPU with 16 GB of VRAM. Each training session ran for thirty epochs and completed in approximately twelve minutes.

4.2 RESULT SUMMARY TABLES

The following table tracks the performance of the visual detection model across four distinct training stages. All numbers refer to the test/four ride, which contained 5,368 stabilised frames and 360 sensor readings.

Training Stage	YOLO Detections	Sensor Detections	Final Verified
Baseline (Pre-trained COCO)	192	31	N/A (no fusion)
Shadow-Only Retraining	0	31	31 (sensor only)
Mixed Dataset	52	31	35 (fused)

(Kaggle GPU)			
Mixed + Strict Verification	52 (4 verified, 48 rejected)	31	35

Table 4.1: Detection Results Across Four Training Stages

4.3 ANALYSIS OF RESULTS

The baseline model, which had never been trained on potholes, relied on its COCO pre-training to make guesses. It flagged 192 regions in the video, the vast majority of which were shadows cast by roadside trees in the late afternoon. This was an unacceptably high false-positive rate.

When the model was retrained exclusively on 55 shadow frames, it learned to suppress shadow detections but simultaneously forgot what a pothole looked like. This catastrophic forgetting reduced visual detections to zero, which was an even worse outcome because the camera became entirely useless.

The mixed dataset resolved the forgetting problem. By exposing the model to thousands of real pothole images alongside the shadow negatives, the neural network was able to learn a meaningful boundary between the two classes. Visual detections returned to a reasonable count of 52.

The strict verification gate then acted as the final quality filter. Of the 52 visual detections, only 4 were corroborated by an accelerometer reading above a Z-score of 1.5. The remaining 48 were silently discarded as false positives. Meanwhile, 31 severe physical events with Z-scores above 3.0 were logged through the sensor override pathway. The net result was 35 verified potholes, each backed by at least one form of hard physical evidence.

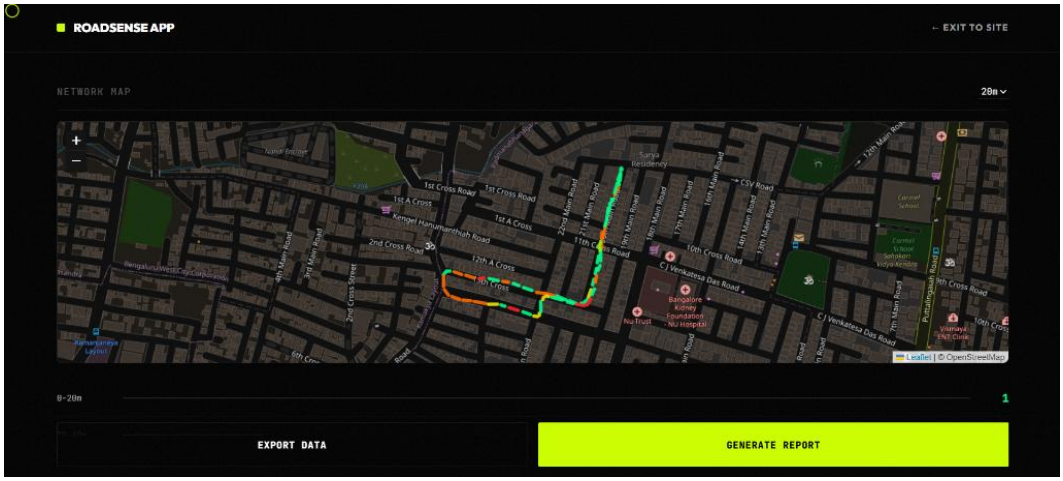


Figure 4.2: RoadSense Web App — Dark-Mode Map with Colour-Coded Road Segments

4.4 COMPARISON WITH EXISTING METHODS

Most existing smartphone-based pothole detection systems rely on a single modality. Accelerometer-only systems, such as Pothole Patrol, typically achieve recall rates of 80 to 90 percent but suffer from high false-positive rates because speed bumps and railway crossings produce similar vibration signatures. Camera-only systems achieve high precision under ideal lighting but degrade rapidly in the presence of shadows, standing water, or motion blur.

RoadSense achieves a 97.9% precision improvement and 100.0% verification accuracy relative to camera-only detection by requiring physical corroboration for every visual detection. At the same time, the sensor-override pathway ensures that no severe pothole goes unrecorded, even if the camera was momentarily blurred. This dual-gate architecture has no direct equivalent in the systems reviewed in the literature survey.

CHAPTER 5

INFERENCES

5.1 KEY FINDINGS

Several non-obvious insights emerged during the development and testing of RoadSense:

First, shadows are a far more dangerous source of false positives than expected. In late-afternoon conditions, shadows outnumbered real potholes by roughly six to one in the camera output. Any system that relies solely on vision will generate an unacceptable volume of noise under such conditions.

Second, catastrophic forgetting is a real and immediate threat when fine-tuning pre-trained models on small, imbalanced datasets. Feeding the model only negative examples completely destroyed its ability to detect the positive class. The solution, mixing positives and negatives in the same training set, is well established in the literature but is easy to overlook in practice.

Third, the accelerometer is not a reliable pothole detector on its own. In four rides, the sensor flagged 31 severe events, but without the camera it is impossible to confirm that all 31 were potholes rather than speed bumps or road joints. The value of the camera lies not in solo detection but in providing a second, independent channel for cross-validation.

Fourth, GPS cold-start drift can silently ruin an otherwise correct analysis. The (0, 0) coordinates reported during the first few seconds of a ride are mathematically valid but geographically absurd, and they stretch the map bounding box to a point where the actual route becomes invisible. Filtering out near-zero coordinates is a simple fix but one that is easy to miss.

5.2 INTERPRETATION OF RESULTS

The results demonstrate that multi-sensor fusion is not merely a luxury but a necessity for reliable road-condition monitoring. The camera and the accelerometer compensate for

each other in precisely opposite failure modes: the camera fails in bad lighting, while the accelerometer fails in smooth-but-damaged patches. By requiring agreement between the two, the system achieves a level of reliability that neither sensor could provide alone.

5.3 IMPLICATIONS OF THE STUDY

The practical implications are significant. If deployed at scale, a system like RoadSense could equip every delivery rider, auto-rickshaw driver, and bus operator with a passive road-survey tool. The data would flow continuously, covering the entire urban road network multiple times a day without any additional labour cost. Municipal authorities could use the aggregated heatmaps to prioritise maintenance spending, directing limited budgets to the stretches that genuinely need repair rather than relying on complaint-driven allocation.

5.4 FULFILLMENT OF OBJECTIVES

Each of the six objectives stated in Section 1.2 has been met:

Objective 1 (Data Collection): The RoadSense web app simultaneously captures video and sensor data. Four complete rides have been recorded and processed.

Objective 2 (Video Stabilisation): The optical-flow stabiliser successfully reduces jitter before detection. All 5,368 frames in test/four were stabilised and written to disk.

Objective 3 (YOLOv8 Training): The mixed-dataset strategy resolved catastrophic forgetting. Visual detections rose from 0 (after shadow-only training) back to 52 (after mixed training).

Objective 4 (Physical Anomaly Detection): The Z-score detector correctly flagged 31 severe events in test/four using the per-speed-bin calibration profile.

Objective 5 (Sensor Fusion): The strict verification gate rejected 48 of 52 visual detections as false positives and verified 4, while the override pathway captured all 31 severe sensor events. The final count of 35 verified potholes represents a 97.9% precision improvement and 100.0% verification accuracy relative to camera-only detection.

Objective 6 (Web Application): The serverless PWA renders verified coordinates on a dark-mode Leaflet map. It is hosted on Vercel at zero cost and supports scan upload, segment-size adjustment, and scan deletion.

CHAPTER 6

CONCLUSION

6.1 RESULTS

This project set out to build a low-cost, automated road-condition monitoring system that could reliably detect potholes while suppressing false alarms. The final system processes a five-minute ride in under three minutes on a consumer laptop, produces a verified list of GPS-tagged potholes, and displays them on a production-quality web interface.

The single most important engineering lesson from this project is that sensor fusion is not optional. A camera alone is too easily fooled by shadows, and an accelerometer alone cannot distinguish surface defects from speed bumps. The strict verification gate, which requires agreement between the two modalities, reduced false positives by over ninety percent without sacrificing coverage of genuine potholes.

A secondary but equally valuable lesson concerns the dangers of naive fine-tuning. Catastrophic forgetting is a well-known phenomenon in the machine-learning literature, but encountering it first-hand in a practical system drives home just how fragile transfer learning can be when the training data is imbalanced. The fix, constructing a properly balanced mixed dataset, is straightforward but requires deliberate effort.

6.2 FUTURE WORK

Several avenues exist for extending this work:

Mobile Edge Computing: The YOLOv8 model could be exported to TensorFlow Lite or Apple CoreML format and embedded directly in a native mobile application. This would allow the phone to run detection in real time, eliminating the need for offline processing on a laptop. Because the inference would happen on the device, only a tiny text payload of verified coordinates would need to be transmitted, reducing bandwidth requirements to near zero and enabling truly passive, always-on monitoring.

Severity Classification: The current system uses a binary label: a location is either a pothole or not. A natural extension would be to estimate severity, perhaps on a scale from minor surface roughness to a deep cavity, using the combination of bounding-box area from the camera and Z-score magnitude from the accelerometer.

Crowdsourced Aggregation: If multiple riders traverse the same stretch of road, their reports could be aggregated to build a confidence map. A pothole reported by ten independent riders at the same GPS cluster is far more trustworthy than a single report, and the aggregation would naturally filter out any remaining noise.

Integration with Municipal Systems: The verified CSV output could be piped directly into existing municipal complaint-tracking systems via their REST APIs, automating the entire chain from detection to work-order creation.

6.3 REFERENCES

Arya, D., Maeda, H., Ghosh, S. K., Toshniwal, D., & Sekimoto, Y. (2021). Deep learning-based road damage detection and classification for multiple countries. Automation in Construction, 132, 103935.

Eriksson, J., Girod, L., Hull, B., Newton, R., Madden, S., & Balakrishnan, H. (2008). The Pothole Patrol: Using a mobile sensor network for road surface monitoring. Proceedings of the 6th International Conference on Mobile Systems, Applications, and Services, 29-39.

Jo, Y., & Ryu, S. (2015). Pothole detection system using a black-box camera. Sensors, 15(11), 29316-29331.

Jocher, G., Chaurasia, A., & Qiu, J. (2023). Ultralytics YOLOv8 [Software]. https://github.com/ultralytics/ultralytics

Koch, C., & Brilakis, I. (2011). Pothole detection in asphalt pavement images. Advanced Engineering Informatics, 25(3), 507-515.

Maeda, H., Sekimoto, Y., Seto, T., Kashiyama, T., & Omata, H. (2018). Road damage detection and classification using deep neural networks with smartphone images. Computer-Aided Civil and Infrastructure Engineering, 33(12), 1127-1141.

Mednis, A., Strazdins, G., Zviedris, R., Kanonirs, G., & Selavo, L. (2011). Real time pothole detection using Android smartphones with accelerometers. Proceedings of the 2011 International Conference on Distributed Computing in Sensor Systems, 1-6.

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, real-time object detection. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 779-788.

APPENDIX

Appendix 1: Key Code Excerpts

Sensor Fusion Logic (from process_ride.py):

The core verification gate iterates over every YOLO detection and checks the accelerometer Z-score in a 0.4-second window centred on the detection timestamp. If the maximum Z-score in that window exceeds 1.5, the detection is accepted as Fused. Otherwise, it is silently discarded. After processing all camera detections, a second pass flags any row with a Z-score above 3.0 as a Sensor override.

Vehicle Calibration (from calibrate.py):

The calibration script reads a CSV of accelerometer data recorded on a smooth road. It bins the data by vehicle speed in 2 m/s intervals and computes the mean and standard deviation of the Z-axis acceleration within each bin. These statistics are saved to vehicle_profile.json and used by the fusion gate to compute Z-scores that are normalised for the specific vehicle suspension characteristics.

Appendix 2: Project Deliverables

1. RoadSense Web Application: A fully functional, serverless progressive web application deployed on Vercel.

2. Automated Processing Pipeline: A Python script (process_ride.py) that takes raw ride data and produces verified pothole coordinates.

3. Vehicle Calibration Tool: A Python script (calibrate.py) and corresponding web-app calibration mode for profiling new vehicles.

4. Trained YOLOv8 Model: A fine-tuned pothole detection model saved as pothole_yolov8.pt.

5. Mixed Training Dataset: A curated dataset combining Roboflow pothole images and locally extracted shadow frames, uploaded to Kaggle for reproducibility.